Implementation and Evaluation of a Tightly-Synchronized WiFi-based Long Distance Network for Rural Area Connectivity

Nurzaman Ahmed, Md. Iftekhar Hussain

Department of Information Technology
North-Eastern Hill University, Shillong, India
nurzaman713@gmail.com, ihussain@nehu.ac.in

Abstract

The computation delays in cloud computing is a concern for smart and services for Internet of Things (IoT) applications and analytic. With the growing network size and application domains, these are increasing tremendously. In this case, a low latency and high capacity backbone network is essential. WiFi-based Long Distance (WiLD) networks have proven to be a suitable solution for extending connectivity towards rural areas. However, due to the long distance links connected by directional antennas, high propagation delays and interference exist. An efficient TDMA-based MAC protocol for multi-hop WiLD networks called 2C has improved end-to-end performance using an interference-aware node coloring algorithm. In this paper, we have implemented 2C MAC protocol over real-life testbed. The implementation incorporates different features of 2C such as tight synchronization over multi-hop and 2C color scheduling based data transmission. Using spatial reuse mechanism of 2C, the implemented testbed shows better performances in terms of saturation throughput and end-to-end delay than other relevant MAC protocols. We also analyze the device to cloud latency and energy consumption in an IoT environment.

Keywords: 2C, WiLD Network, TDMA, Atheros Driver, Multi-hop Synchronization

1. Introduction

Last couple of decades have eye-witnessed a huge advancement of Information and Communication Technology (ICT) infrastructure in order to provide Internet for wide range of social, political, and economic benefits [15]. Unfortunately, rural areas, especially in developing countries are undervalued from this vital infrastructure which has resulted in digital divide [10]. A viable solution to this problem is to use low cost and easily deployable WiFi-based Long Distance (WiLD) networks in extending Internet connectivity to such areas [18].

In WiLD, due to the use of high gain directional antennas, interference and several other relevant issues are common. In such case, the protocols need be design for optimally utilizing the network resources to achieve better performances. The Medium Access Control (MAC) protocols plays a tremendous responsibility to utilizing the bandwidth of available wireless links. Raman et al. [19] achieves an interference free scheduling of transmission ($SynTx$) and reception ($SynRx$) for different radios located in the same tower termed as Simultaneous Synchronous Operation ($SynOp$). $SynOp$ based MAC protocols like 2P [19], WiLDNet [18], and JazzyMAC [16] use a special packet called *marker packet* to switch its mode of operations (i.e., SynTx or SynRx). A node having the *marker packet* can only transmit for a given duration of time, hence the overall network performance is very low especially in saturated scenario.

A TDMA-based MAC protocol called 2C (2-Color) [13] utilizes the SynOp operation and proposes a spatial reuse mechanism to improve WiLD network performance. The 2C MAC protocol logically organize the overall WiLD network into a tree considering gateway node as root. The 2C MAC enables SynOp to all the nodes in different levels of the tree at all times. In the synchronization phase of the protocol, all the nodes in the network synchronized with respect to same time and color. The color assigned to a node represents either SynTx or SynRx operation of SynOp at a particular time. The time and color synchronization of 2C achieve interference free schedule of links and optimal slot reuse by the nodes. Finally, in the data transmission phase, all nodes checks its color information (RED or GREEN) and switch its transmission mode of operation accordingly.

This paper describes the implementation of *2C* [13] over commodity WiFi hardware using the open source Atheros driver [2]. The software MAC (SoftMAC) [17] has developed *net80211* and *mac80211* open source Wi-Fi driver module to provides end user code accessibility. Further, [21] and [9] explore this module for TDMA implementation over in build CSMA/CA based MAC. We have customized the 802.11 header format to generate control, node join and data packets. To achieve that, we first disabled the properties of CSMA/CA MAC protocols and then proceed with implementing with TDMA system. We have modified the ath9k driver for our implementation. Features like control packet generation, timer, queue, TDMA, multiple interfacing and multi-hop communication, 2C synchronization, and sending multiple packets in a slot have been incorporated for the implementation. Our implementation achieves high performance in terms of throughput and delay by providing support for spatial reuse over 2P protocol.

The rest of the paper is organized into four sections. Section 2 discusses the related works. In Section 3, the testbed implementation of our 2C MAC is discussed. It includes the assumptions, modifications, and integration to Atheros 9k (Ath9k) driver. Section 4 gives the performance evaluation of 2C MAC protocol. It also provides a comparison of 2C and 2P protocols in the driver level. Finally, Section 5 concludes our paper.

2. Related Works

[22]

A list of TDMA-based MAC protocols are found in the literature addressing different issues of WiLD networks. The performance analysis of 2P MAC protocol [19] is carried out in an

indoor environment. 2C protocol is implemented on HostAP driver [1]. A testbed setting as (A)$i_1 \Leftrightarrow i_1$(B)$i_2 \Leftrightarrow i_1$(C) is considered for testing the performance of UDP traffic which is generated from A to C through the intermediate node B. This testbed consist of four interfaces and relay node B having maximum 2 interfaces (i_1 and i_2). With the use of SynOp mechanism, the performance of 2P is improved over conventional CSMA/CA MAC protocol. Further, to enhance the TCP performance of 2P, WiLDNet [18] has incorporated an adaptive loss-recovery mechanism that uses a combination of Forward Error Correction (FEC) and bulk acknowledgments to significantly reduce the packet loss rate. Atheros chipset based driver, MadWiFi is used for the implementation.

JazzyMAC [16] uses a dynamic TDMA slot scheduling mechanism for nodes having different traffic demands. The simulation analysis of JazzyMAC in a Java based network simulator [14] achieved superior throughput over static TDMA approaches like 2P [19]. However, token exchange in it still a bottleneck for overall network performance. For the first time, Dhekne et al. [11] developed TDMA MAC over multi-hop network in MadWiFi driver. Further, LiT MAC [20]) achieved better multi-hop time synchronization using routing mechanism. The performance of the scheme is taken over a indoor and outdoor testbed setup. However, as the protocols assign TDMA slots to each nodes to communicate, with the increase in number of nodes, the transmission time for a node decreases. Table 1 shows the evaluation details of related works.

To facilitates applications such as e-learning, tele-medicine, e-governance etc., for rural areas, the WilD networks need to provide some sort of QoS guarantees. The simulation analysis of 2C protocol shows delay and throughput performance improvements of WiLD network by incorporating optimal slot reuse mechanism. However, actual implementation of 2C protocol still need to be done.

3. 2C MAC Implementation

We have implemented 2C MAC protocol over commodity WiFi hardware using the open source ath9k [2] driver. In this work, firstly we have highlighted the salient features of our implementation technique. Furthermore, towards the evaluation of the implementation in various dimensions: multi-hop synchronization, time granularity of scheduling, and achievable throughput for UDP & TCP flows have also discussed. The implementation achieves high performance in terms of throughput and delay by providing support for spatial reuse.

3.0.1 TDMA Frame Format and Packets Used

The TDMA frame and packet formats are defined according to the requirements of the 2C protocol.

TDMA Frame

The 2C protocol defined TDMA frame as a combination of control slots, contention slots, and data slots. Control informations are transmitted using *control slots*. *Contention slots* are used

Table 1: Summary of related works

Protocol	**Evaluation**	**Details**
2P [19]	**Simulaton-** NS-2, **Implementation-** HostAP v0.2.4 driver on Linux v2.4.20	Prism2 card based HostAP is an open source WiFi driver
WilDNet [18]	**Implementation-** MadWiFi driver on Linux v2.4.26	MadWiFi works over Atheros-based chipsets in Linux system
JazzyMAC [16]	**Simulation-** DTN Simulator	Java-based network simulator developed by Jain et al.[14]
Dhekne et al. [11]	**Implementation-** MadWiFi driver on Linux	MadWiFi works over Atheros-based chipsets in Linux system
LitMAC [12]	**Implementation-** MadWiFi driver on Linux v2.6	Linux Openwrt Kamikaze v8.09, and Openwrt Backfire v10.03 is used
2C [13]	**Simulaton-** NS-2	An open source and discrete even driven network simulator

bu the non-root nodes to send node joing request. Finally, data transmission occur through *data slots*. Fig.1 shows the proposed TDMA frame for 2C implementation.

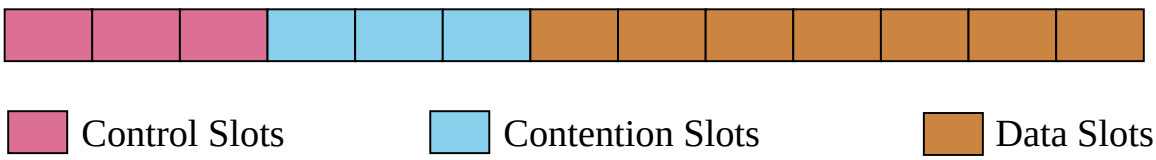

Figure 1: TDMA Frame Format

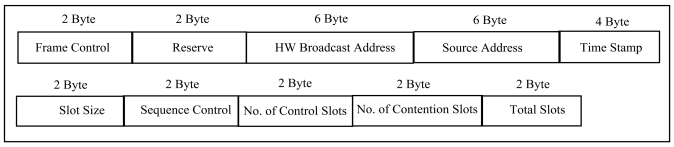

Figure 2: Control Packet

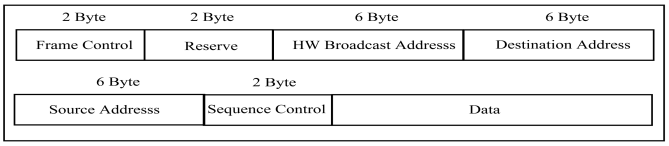

Figure 3: Data Packet

Packet Types

3.1. Packet Type

We have customized the 802.11 header format to create three types of packets viz., control, node join, and data.

3.1.1 Control Packets

Control packets carry control information from the root node to the relay and leaf nodes. Different fields in the control packet are shown in Fig 2.

- Header: It holds multi-hop time synchronization information.
- Frame Info: It contains the frame information such as start and end time of TDMA slots, schedule of control and contention slot, and global time information.

3.1.2 Data Packet

It is the actual data which needs to be transmitted from source to destination. Data packet contains-

- Data header: It contains frame control, address and checksum informations.
- Payload: For MAC layer, the data received from network layer is the payload. The MAC header is added/removed with the payload while processing.

3.1.3 Node Join Packet

It is a type of packet which is transmitted during a contention slot. It is basically used by nodes which want to join the network. The Node Join Packet consists of additional fields:

- Parent Index: It is the id of the node to which the node wants to join.
- Node index: It is the id of the node which wants to join the network.

3.1.4 Hardware Platform

We have used 2-hop outdoor testbeds for the performance evaluation. Our main goal is to implement a tightly synchronized multi-hop TDMA MAC on commodity WiFi hardware, to take the advantage of the low-cost benefits of the same. This involves resolving some important issues. We have modified the ath9k driver for our implementation. This driver works for the popular Atheros WiFi chipsets.

We have used two types of router boards- Mikrotik RB433AH [6] and Mikrotik RB411 [5]. Considering different hardwares, a testbed setup can be seen in Fig. 4.

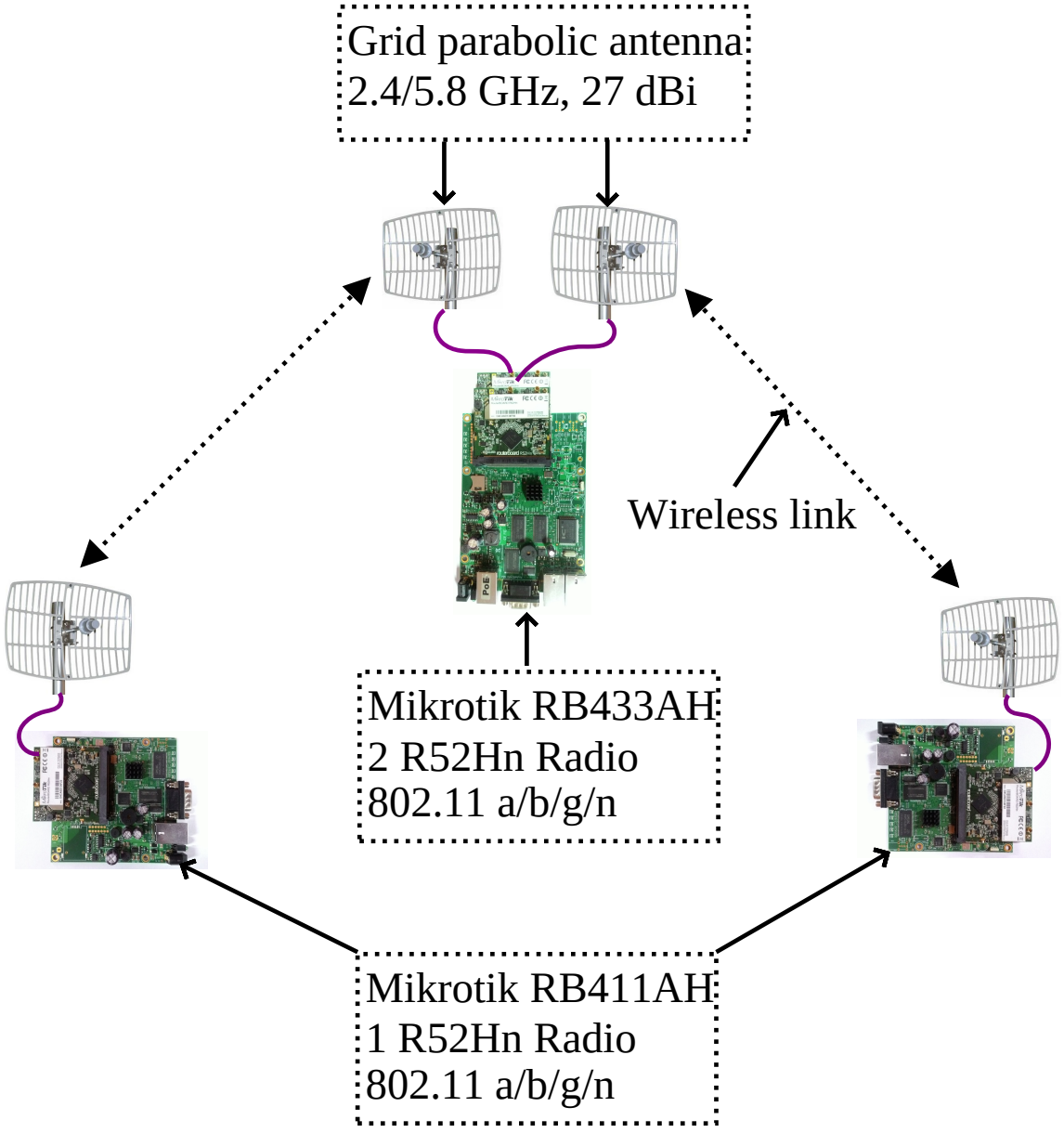

Figure 4: The testbed setup hardwares with configurations

As the router board RB433AH have multiple radios, it is used as a relay node and RB411 is as leaf node. The latest OpenWrt development Chaos Calmer [7] which is based on Linux

kernel v3.14 is run on these boards.

3.2. Modification to Ath9k Driver

The socket buffer or sk_buff (skb) [8] is the most fundamental data structure in the Linux networking code. Every packet sent or received is handled using this data structure. It contains all the data and layers information. Some more structures sub_if_data (sdata) and channel (chan) are used in case of interface and channel related decision making respectively. The following steps were followed during our implementation phase.

3.2.1 Disabling the Features of CSMA/CA

The ath9k device driver implements CSMA as MAC layer protocol while for the TDMA implementation we need a packet transmitter and receiver, with strict control over packet transmission timing. Given such requirements, we first need to disable CSMA and then proceed with implementing the TDMA system. They are as follows-

- Disable MAC level ACK's
- Disable RTS/CTS
- Sending custom frame format (no 802.11 frame)
- Disable transmission back-off
- Disable virtual carrier sensing
- Disable CCA mechanism

We also found that, when we set the card in monitor mode, it disables MAC layer ACK, RTS/CTS mechanism and allows us to send custom frames on air which directly achieves first three task in process of disabling CSMA mechanism. In order to disable virtual carrier sensing and transmission back-off, we set the bit positions used in the 802.11 MAC Header format for Network Allocation Vector (NAV) or reset the duration ID field to zero in the TDMA MAC Header. We have turned off the bulk acknowledgment mechanism by specifying each packet transmitted as broadcast packet. For sending the data packet we have used a function *sendData* and for receiving *recvData* function is used.

3.2.2 Communication in Monitor Mode

Monitor mode allows us to send raw packets without extensive modification. Since, it is not designed for two way communication, our first task is to establish two way communication in monitor mode. The Transmission(Tx) and Reception (Rx) paths are shown in Fig. 5.

Tx path in Monitor Mode

(i) The kernel first calls *ieee80211_monitor_start_xmit (sk_buff, net_device)* in tx.c by passing a skb and dev and calls the *ieee80211_xmit*

(ii) *ieee80211_xmit(skb, sdata)* in tx.c takes the skb and sdata as input and calls *ieee80211_tx*

(iii) *ieee80211_tx(sdata, skb, txpending)* in tx.c takes sdata, skb and a boolean flag and then call *ieee80211_tx*

(iv) *ieee80211_tx(local, skb, led_len, sta, txpending)* in tx.c which in turn calls *ieee80211_tx_frags*

(v) *ieee80211_tx_frags(local, vif, sta, skbs, txpending)* in tx.c calls *drv_tx*

(vi) *drv_tx (local,skb)* in driver-ops.h is the entry point to the driver.

Rx path in Monitor Mode

(i) While receiving a packet, the hardware sends an interruption that ath9k has previously registered. The function in charge of handling the interrupt seems to be irqreturn_t ath_isrin (main.c). This function discovers the type of interrupt and asks the kernel to schedule the execution of a tasklet, ath9k_tasklet in main.c. This function in turn calls the receive tasklet *ath_rx_tasklet*

(ii) ath_rx_tasklet(recv.c) obtains the frame header and the current Timing Synchronization Function (TSF) value and record info about received packet in struct ieee80211_rx_status and insert received data in the receive buffer. After that a new skb is created to contain the receive data and passes the skb to *ieee80211_rx* which is the entry point to mac80211 module.

(iii) *ieee80211_rx_handle_packet* in rx.c calls the *ieee80211_rx_monitor* to remove Frame Check Sequence (FCS) and Radiotap if any.

(iv) *ieee80211_rx_monitor* in rx.c returns a cleaned up skb to the called function or if the interface is in monitor mode it calls *netif_recive_skb*

(v) *netif_receive_skb* is the entry point to the kernel.

The main changes that takes place in our implementation lies in *ieee80211_monitor_start_xmit* function in transmission side and *ieee80211_rx_monitor* function in reception side.

Though the packets from network layer are transmitted in air without any manipulation but in the receiver side a 34 bytes radiotap header is attached to every packet which makes the packet unrecognizable to the network layer on the receiver side. For this, we have created a *recvData* function which removes the 34 byte radiotap header, 802.3 header, 2 byte protocol id, and 4 byte CRC from the tail and return the modified packet to *ieee80211_rx_monitor* function which in turn passes it to *netif_receive_skb* function. A Tx-Rx process is shown in Fig. 5 considering primary functions communicating with the network layer.

Though the packet is transferred to the network layer but it does not reply any ping packet. The reason behind it is that the packet gets corrupted during transmission as it doesnâĂŹt have any 802.11 header. To solve it, we made a header format similar to 802.11 header format keeping the position of some fields intact. Also specifying a frame as retry frame in the header prevents the frame from being modified by the hardware during transmission. We have tested

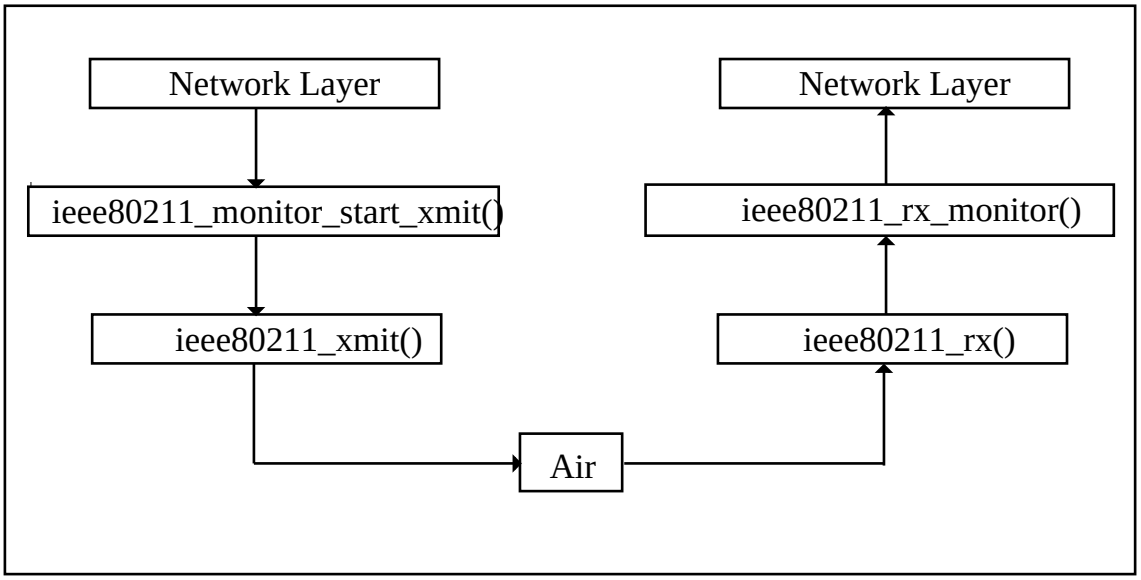

Figure 5: Tx and Rx path in ath9k

the communication using ping packet and after the changes being made we were able to ping two boards in monitor mode.

3.3. Integration to Ath9k Driver

3.3.1 Control Packet Generation in MAC Layer

Control packets are generated to tightly synchronize the Tx-Rx operation of the participated nodes and to know the neighbor interfaces information by an interface in a node. Custom control header informations are attached to the control packets that arrived to the MAC layer from network layer, so that the receivers can distinguish the valid packets. We have created *ipk* packages which is further been installed in OpenWrt. The installed package is run whenever a control packet need to be generated for a particular interface. We capture a previous skb and sdata(extracted from net_device) from the function *ieee80211_monitor_start_xmit* and store them in three global variables globalskb, globalsdata and globalchan respectively. The function *sendCtrl* takes the responsibility to generate control packets. While generating a control packet, the function *sendCtrl* creates a copy of the globalskb, globalsdata and globalchan in skb, sdata and chan respectively. We call the function *add_tdma_control_Header* which flushes the data field of the skb and then inserts the appropriate control header into it. Then the function *ieee80211_xmit* is called passing the skb containing the control data and the sdata.

3.3.2 Timer Implementation

In the Linux kernel, time is measured using global variable named jiffies [4]. But we found that the kernel jiffies provide tick of 1 ms granularity which is not suitable for our current implementation, as it requires granularity in microsecond range. So, we use a timer module called High Resolution Timer (HRT) [3] which gives granularity in nanoseconds. We have

implemented two HRTs. One is the slot_timer which is used to generate the time slots. The handler of this timer calls the *startClock* function when the timer expires which updates or increments the variable jiffy. The granularity of the timer is 100 microsecond. The *startClock* function in turn calls the *sendData* function during each transmission slot. Another timer is the send_timer which is used to keep track of the transmission of a packet. The handler of this timer calls the *sendData* function only when the timer expires. The basic need of send_timer is to send multiple packets in one transmission slot.

3.3.3 Time Synchronization

The efficiency of a TDMA system relies on how tightly the nodes are synchronized. The synchronization process is done periodically to overcome the loss of synchronization due to clock drift. Also guard time is used between two slots to avoid overlap of transmissions. Every node maintains a variable called jiffy which gets incremented as per the given timer resolution. The time synchronization process is initiated by the root node by sending the current value of jiffy in the control packets as time stamp. After receiving the control packet, the receiver node extracts the time stamp and updates its jiffy with that value. Here we have assumed the propagation time to be negligible.

Algorithm 1 Time Synchronization

```
1: current_jiffy ← Current value of jiffy as timestamp
2: recv_jiffy ← Receive value of jiffy as timestamp
3: max_delay ← Empirically calculated maximum delay offset
4: if abs(recv_jiffy-current_jiffy ) < max_delay then
5:     current_jiffy= recv_jiffy+(max_delay + (recv_jiffy-current_jiffy))/2
6: else
7:     current_jiffy= recv_jiffy+max_delay
8: end if
```

As discussed in the time synchronization Algorithm **??**, the process maintains an empirically calculated maximum delay offset. If the difference between the current jiffy and received jiffy is less than the default value, we add the average of maximum delay offset and the value of the difference between the current jiffy and received jiffy and add it with the received jiffy. This result is then used to update the current jiffy value. If the difference between the current jiffy and received jiffy is more than the maximum delay offset, we add the maximum delay offset with the received jiffy and update the current jiffy. The data flow of the implemented TDMA protocol can be seen in Fig. 6.

3.3.4 Queuing at MAC Layer

For queuing outgoing packet, we maintain a software queue using linked list. We used two pointers to the tdma_queue. One is tdmahead and the other is tdmatail to point to the front and rear of the tdma_queue. The packets from the network layer are handed to *ieee80211_monitor_start_xmit* function passing skb and network device information. After that we queue both the skb and the sdata (extracted from the network device information) in the tdma_queue if the queue length is less than MAX_QUEUE_SIZE. If not, we drop the packet.

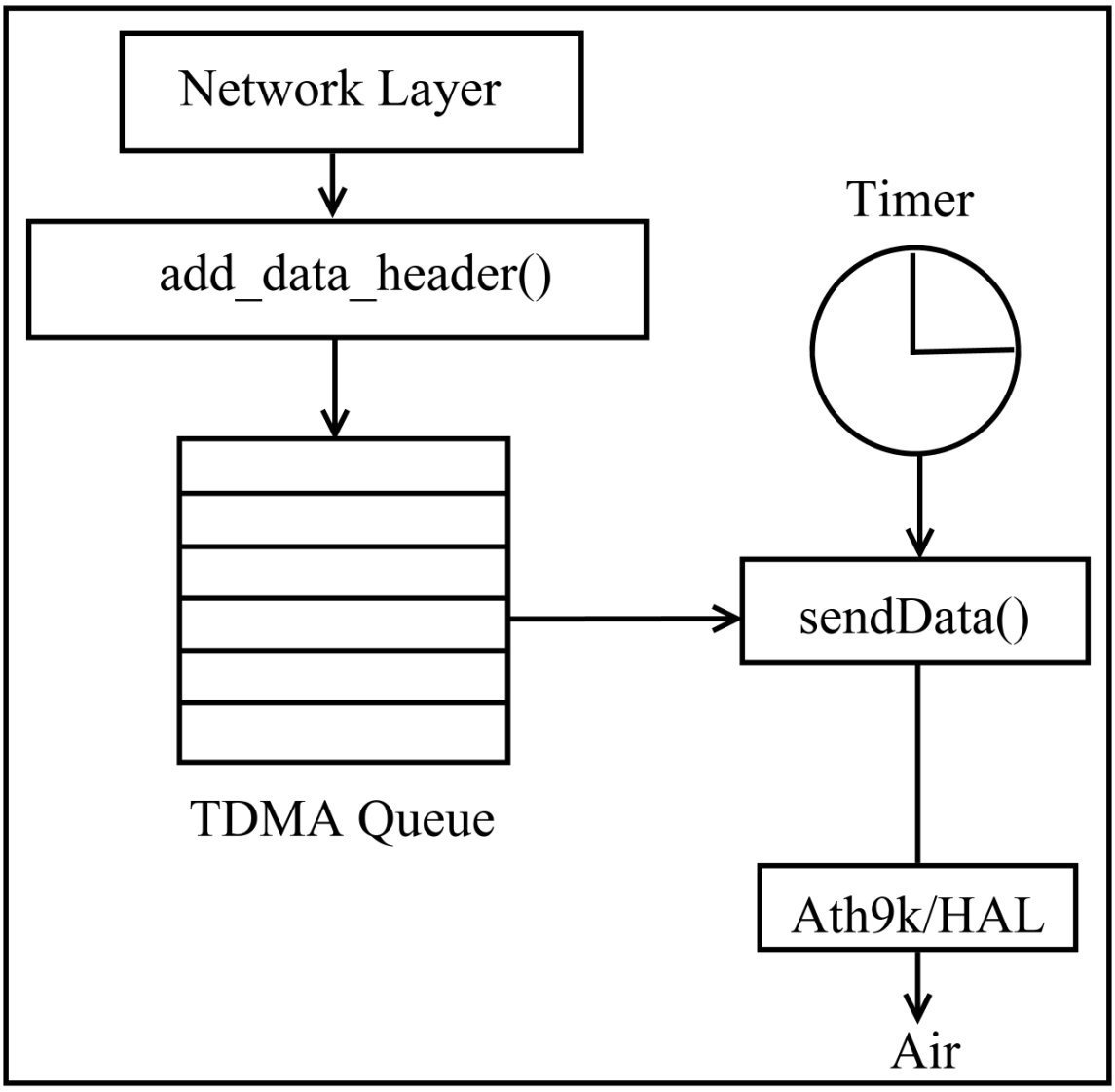

Figure 6: TDMA MAC Protocol in ath9k

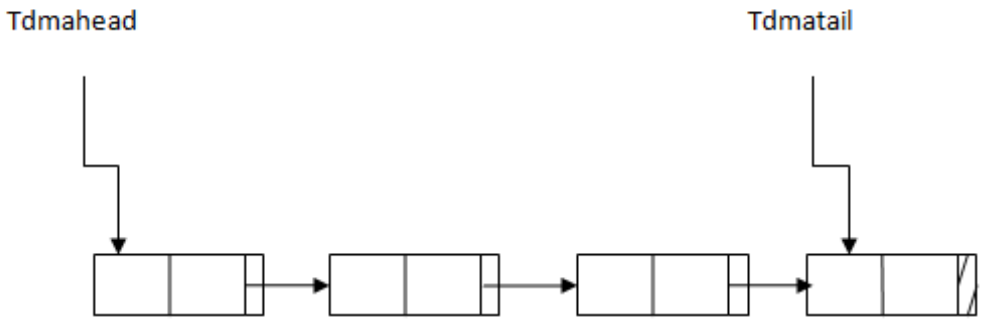

Figure 7: TDMA Queue example with four elements

The function *sendData* (which is used for sending the data) when called will extract a packet from the queue and passes it to *ieee80211_xmit* which will finally sends the packet to the driver for sending it in air. A link list queue implementation example is shown in Fig. 7.

3.3.5 Sending Multiple Packets in a Slot

We have discussed the queuing and timer implementation in above sections. Using the send_timer and queue maintained in MAC layer, we send multiple packets in a single slot. Algorithm 2 finds remaining time of slot after transmission of first packet and used it for sending more packets.

3.3.6 Multiple Interfacing

To achieve multi-hop synchronization and communication, we incorporate multiple interface support. Using control packets and global variables one interface informs name to its neighbor interfaces. Along with the interface information, they able to share the channel which leads to the uses of multiple channels. Similar to the generation of control packets from *ieee80211_monitor_start_xmit* function, we capture skb, sdata, and chan store them in three global variables globalskb, globalsdata and globalchan respectively. The Algorithm 3 shows the decision taken by a relay node to process a packet.

The relay node checks the destination information (itself or others) from *skb->data* field and if it finds that the packets are not destined to itself, it calls the *forwardData* function to send packets toward the destined node through different interface. The relay node uses the interface and channel information acquired from global variables to choose appropriate interface and channel. User can decide the use of channel (multiple or single) using *iwconfig* command from shell.

3.3.7 2C−Color Synchronization

The root node assigns a color to itself and sends the color as one bit information in the $frame\_control$ field of the TDMA control header in the control packet. We maintain a variable

Algorithm 2 Sending Multiple Packets in a Transmission Slot

```
pkt_time = dataTime(packet_size)
tdmahead ← Pointer to the front of the tdma_queue
tdmatail ← Pointer to the rear of the tdma_queue
queue_len ← Length of the tdma_queue
my_time ← Remaining time of a TDMA Slot
guard_time ← Guard Time for a TDMA Slot
rem_slot_time = my_time - guard_time
if tdmahead then
    if pkt_time < rem_slot_time then
        if tdmahead = tdmatail then
            tdma_queue is empty
        else
            Dequeue packet from tdmahead
            Update tdmahead
            queue_len = queue_len-1
            send(packet)
            my_time= my_time - pkt_time
        end if
    end if
end if
procedure dataTime(packet_size)
    return(packet_size/link_bandwidth)
end procedure
```

Algorithm 3 Packet Interfacing at Relay Node

```
packet_skb ← Recieved data for Network Layer
iface_rdev ← Interface through packet_skb recieved
iface_sdev ← Interface through packet_skb need to be send
if packet_skb=broadcast_type then
    recvData(packet_skb)
    forwardData(packet_skb, iface_rdev)
else
    if packet_skb=relay_type then
        forwardData(packet_skb, iface_rdev)
    else
        recvData(packet_skb)
    end if
end if
procedure recvData(packet_skb)
    skb ← packet_skb with removed header information
    return(skb)
end procedure
procedure forwardData(packet_skb, iface_rdev)
    Send packet_skb through iface_sdev
end procedure
```

named *color* which is used to keep track of the color assigned to a node. We use two color codes-RED and GREEN. The color RED is used to indicate transmission and the color GREEN is used to indicate reception. The reverse assignment also can be used for indication. Algorithm 4 is run by different nodes for properly synchronize the color of it.

Algorithm 4 Color Synchronization

```
color ← (RED, GREEN)
mac_state ← (Tx, Rx)
if Control Slot then
    if Root Node then
        SEND(Control_packet(color))
    else
        RECEIVE(Control_packet(color))
        if color = RED then
            color = GREEN
        else
            color = RED
        end if
    end if
else if Data Slot then
    if color = RED then
        mac_state=Tx
        SENDDATA()
    else
        mac_state=Rx
    end if
end if
```

In the color synchronization process, the root node sends its color in the control packet to the other node. On the other hand, the receiver of the control packet extracts the color information from the packet and assigns the inverse color of it. If the color is GREEN than it assigns RED to itself or set the color variable to RED (i.e., 0) and vice versa. At the starting of Data slot, the MAC protocol checks the color variable whether it is RED or GREEN, if it is RED then we call the function *sendData* to transmit data and if the color is GREEN, we defer our transmission to the next slot until the color becomes RED. At the end of the slot we invert the color of the nodes, i.e., the node colored RED becomes GREEN and the node colored GREEN becomes RED. Using the color synchronization process the nodes schedule their transmission independently without affecting the transmission of the other nodes.

4. Performance Evaluation

A two hop linear topology connected by point-to-point high gain directional antenna is used to analyze the performance of the proposed protocol. UDP and TCP traffics are generated using the *iperf* tool. Also *Ping* application was used to check the network connectivity and calculate the round trip delay. All the experiments were done in 802.11b frequency range where the bandwidth of the system is 11 Mbps.

Table 2: Experiment Parameters

Parameter Name	**Value**
Traffic types	TCP, UDP
Packet size (UDP)	1470 bytes
Bandwidth	11 Mbps
Antenna type	Grid Parabolic Antenna
Antenna gain	27dBi
No. of nodes (Max.)	3
Guard time	1 ms
TDMA Queue length	200

4.0.1 No. of Hop(s) Vs Saturation Throughput

We conducted experiments for TCP and UDP throughput on linear topology discussed in experimental setup with a slot size of 10 ms. The results for 1-hop and 2-hops are shown in Fig. 8. In order to saturate the link we have used iperf between two machines and set the bandwidth requirement of the UDP connection as 5 Mbps. The default payload of the packets is 1470 bytes. The maximum application throughput that can be obtained on a single 11 Mbps link in a given direction is half of the 11Mbps raw rate, minus the packet header overheads. For unidirectional UDP traffic in 1-hop, we have achieved saturation throughput about 4.55 Mbps and for TCP it is about 4.58 Mbps. But, in 2P [19], the saturation throughput is 3.05 Mbps for UDP traffic. Similarly in 2-hop scenario, maximum throughput achieved in 2C is 4.46 Mbps and 3.78 Mbps for UDP and TCP respectively. But, in 2P, they have achieved maximum 2.81 Mbps for UDP traffic. Hence, our 2C protocol performs better than 2P in case of saturation throughput.

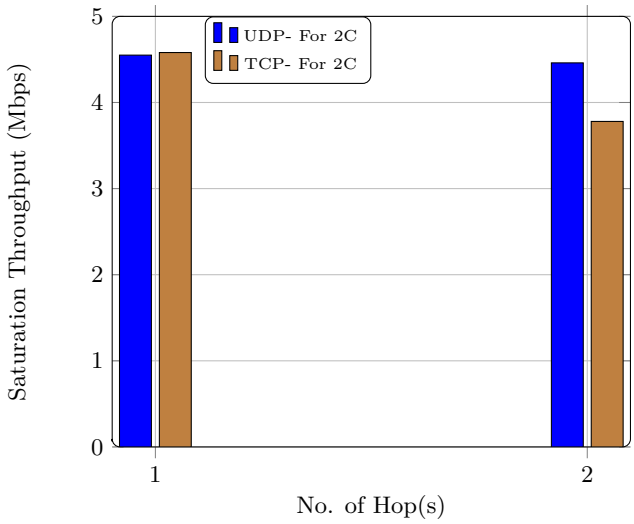

Figure 8: Saturation throughput achieved by the 2C protocol in multi-hop scenario

As 2C protocol guarantees every alternate slot to the nodes, the end-to-end throughput does not get reduced significantly over multiple hops. However, TCP throughput decreases as compared to UDP throughput over multiple hops. This is because of the fact that an increase

in the number of hops increases the round trip delay and end-to-end error probability.

4.0.2 Effect of Throughput on Slot size

The slot size variations have no such impact on the UDP throughput. Since, we do not fragment packets at the MAC layer, an increase in slot size causes lesser overhead. Also, we implemented a small guard band of 1 ms for every slot. As shown in the Fig. 9, UDP performs better with increasing slot size than TCP.

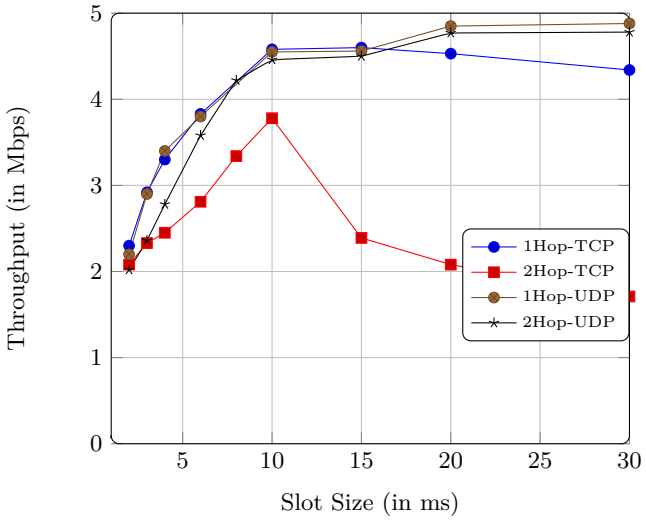

Figure 9: Effect of throughput on slot size

TCP also achieves better throughput but it is adversely affected by increased slot size after a certain limit over multiple hops. This is due to higher end-to-end delay in receiving acknowledgement.

4.0.3 Ping Delay with varying Slot sizes

The delay (Ping turnaround time) is calculated with varying slot sizes. Fig. 10 shows that delay in 2C MAC protocol is well within the delay bound for delay sensitive traffic. A tolerable increase in delay is seen with increasing number of hops. In 2C, the reception and transmission occurs in the consecutive slots which ensures strict end-to-end delay over multiple hops.

5. Conclusion

In this paper, we have implemented multi-hop TDMA based MAC protocol named as 2C. The implementation has achieved different features proposed by 2C MAC protocol over latest open source Atheros based driver. The contributions of this paper can be summarized as follows:

1. The implementation achieved nano-levels of synchronization using a high resolution timer.
2. Multi-hop support using RAW packets for longer distance of communications.
3. It has improved throughput performances by incorporating features of sending multiple packets within a single TDMA slots.

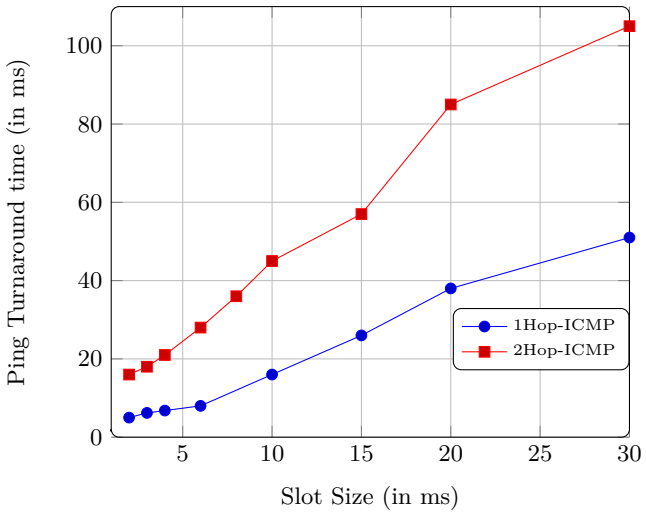

Figure 10: Multihop Delay with varying slot sizes

4. 2C synchronization mechanism efficiently schedule wireless links to avoid interferences and keeps the communication links busy for almost 100% of the time.

5. 2C protocol outperforms 2P in terms of end-to-end throughput and delay. Throughput of CBR and FTP has been enhanced in 2P implementation. Delay performance is also improved radically.

References

[1] About Hostapd. `https://wireless.wiki.kernel.org/en/users/documentation/hostapd`.

[2] ath9k:Atheros Linux Wireless Drivers. `https://wireless.wiki.kernel.org/en/users/drivers/ath9k`.

[3] High Resolution Timers. `http://elinux.org/High_Resolution_Timers`.

[4] Kernel Timer Systems. `http://elinux.org/Kernel_Timer_Systems`.

[5] Mikrotik RouterBoard RB411. `http://wiki.openwrt.org/toh/mikrotik/rb411`.

[6] Mikrotik RouterBoard RB433. `http://wiki.openwrt.org/toh/mikrotik/rb433`.

[7] OpenWrt-Chaos Calmer. `http://wiki.openwrt.org/about/history\#chaos_calmer`.

[8] The Socket Buffer. `http://www.linuxfoundation.org/collaborate/workgroups/networking/sk_buff`.

[9] SS Ahmed, I Hussain, and N Ahmed. Driver level implementation of tdma mac in long distance wifi. In *Computational Intelligence and Networks (CINE), 2015 International Conference on*, pages 80–85. IEEE, 2015.

[10] Neil Anderson. Digital technologies and equity: gender, digital divide and rurality. *Teaching and Digital Technologies: Big Issues and Critical Questions*, 46, 2015.

[11] Ashutosh Dhekne, Nirav Uchat, and Bhaskaran Raman. Implementation and Evaluation of a TDMA MAC for WiFi-based Rural Mesh Networks. *ACM Workshop on Networked Systems for Developing Regions (NSDR '09)*, Oct 2009.

[12] Vijay Gabale, Bhaskaran Raman, Kameswari Chebrolu, and Purushottam Kulkarni. LiT MAC: Addressing the Challenges of Effective Voice Communication in a Low Cost, Low Power Wireless Mesh Network. In *Proceedings of the First ACM Symposium on Computing for Development*, ACM DEV '10, pages 5:1–5:11, New York, NY, USA, 2010. ACM.

[13] Md Iftekhar Hussain, Zaved Iqubal Ahmed, Nityananda Sarma, and DK Saikia. An Efficient TDMA MAC Protocol for Multi-hop WiFi-Based Long Distance Networks. *Wireless Personal Communications*, 86(4):1971–1994, 2016.

[14] Sushant Jain, Kevin Fall, and Rabin Patra. *Routing in a Delay Tolerant Network*, volume 34. ACM, 2004.

[15] Alma Mačiulytė-Šniukienė and Elina Gaile-Sarkane. Impact of Information and Telecommunication Technologies Development on Labour Productivity. *Procedia-Social and Behavioral Sciences*, 110:1271–1282, 2014.

[16] Sergiu Nedevschi, Rabin K Patra, Sonesh Surana, Sylvia Ratnasamy, Lakshminarayanan Subramanian, and Eric Brewer. An Adaptive, High Performance MAC for Long-Distance Multihop Wireless Networks. In *the* 14^{th} *ACM International Conference on Mobile Computing and Networking*, pages 259–270. ACM, 2008.

[17] Michael Neufeld, Jeff Fifield, Christian Doerr, Anmol Sheth, and Dirk Grunwald. SoftMAC-Flexible Wireless Research Platform. In *the* 4^{th} *Workshop on Hot Topics in Networks (HotNets-IV)*, Nov. 2005.

[18] Rabin Patra, Sergiu Nedevschi, Sonesh Surana, Anmol Sheth, Lakshminarayanan Subramanian, and Eric Brewer. WiLDNet: Design and Implementation of High Performance WiFi Based Long Distance Networks. In 4^{th} *USENIX Symposium on Networked Systems Design & Implementation (NSDI, 2007)*, pages 87–100. ACM, 2007.

[19] Bhaskaran Raman and K. Chebrolu. Design and Evaluation of a new MAC Protocol for Long-Distance 802.11 Mesh Networks. In *the* 11^{th} *Annual International Conference on Mobile Computing and Networking (MobiCom '05)*, pages 156–169. ACM, 2005.

[20] Vishal Sevani, Bhaskaran Raman, and Piyush Joshi. Implementation-Based Evaluation of a Full-Fledged Multihop TDMA-MAC for WiFi Mesh Networks. *IEEE Transactions on Mobile Computing*, 13(2):392–406, 2014.

[21] Ashish Sharma, Mohit Tiwari, and Haitao Zheng. MadMAC: Building a Reconfiguration Radio Testbed using Commodity 802.11 Hardware. In 1^{st} *IEEE Workshop on Networking Technologies for Software Defined Radio Networks, (SDR 2006)*, pages 78–83, Sep. 2006.

[22] Xiying Fan Di Liu Peng Li Xi Chen, Chuanhe Huang. LDMAC: A propagation delay-aware MAC scheme for long-distance UAV networks. *Computer Networks*, 144:40–52, 2018.